

人工智能基础与应用 · 大模型实训

课题 02-Transformer 骨架

开题简报 · 交付要求 · 判卷记录

孙承泽 · 能动强基 2501 · 西安交通大学

2026-09-17 生成 · 版本 v1.0

课题 02 开题简报 · Transformer 骨架与训练循环

模块: M0/M1/M3 | 周次: W2-W3 (09-21→10-04) | 算力: T0 纯 CPU, 0 GPU·小时 唐杰作业对应: ① 从零写 Transformer 的后半 | 判分来源: CS336 A1 tests/test_model.py、test_nn_utils.py、test_optimizer.py、test_serialization.py 手写: R2 (Transformer 全部组件) + R3 (训练循环与优化器) + R4 (反向传播验证)

一、研究背景

2017 年《Attention Is All You Need》删掉 RNN, 用自注意力 + 前馈层实现 **O(1) 依赖路径 + 全并行**。此后所有主流大模型都是它的变体, 但**细节已经换了一轮**: 正弦位置编码 → RoPE; LayerNorm → RMSNorm; post-norm → pre-norm; ReLU MLP → SwiGLU; MHA → GQA。本课题要求你把现代版写出来, 而不是复刻 2017 版。

二、研究问题（一句话）

一个“能从零训起来”的现代 Transformer 层, 最少需要哪几个组件? 每个组件去掉之后, 训练会以什么方式失败?

三、攻关线索

1. 形状先行: 写下每一步的张量形状 (含 batch、序列、头数), **先能在纸上跑通**, 再写代码。
2. 注意力的三个动作: 投影出 Q/K/V → 缩放点积 + 因果掩码 → softmax 加权求和并输出投影。哪一步最容易错? 为什么?
3. 位置信息: RoPE 为什么是“旋转 Q/K”而不是“加到输入上”? (讲义 M3 § 三给了恒等式)
4. 归一化与残差: 为什么 pre-norm 比 post-norm 好训? 残差提供了什么“通路”?
5. 优化器: AdamW 里“解耦权重衰减”解耦的是什么? 为什么它比 L2 正则更常用?
6. 稳定性: 梯度裁剪在什么情况下救命? fp16 vs bf16 的差别如何影响 loss 曲线?

四、里程碑

- ☐ M1 手写全部组件: 注意力 (含因果 mask 与 RoPE)、RMSNorm、SwiGLU FFN、残差堆叠、初始化
- ☐ M2 形状与数值判分通过: test_model.py / test_nn_utils.py 全绿
- ☐ M3 手写训练循环 + AdamW + LR 调度 + 梯度裁剪 → test_optimizer.py / test_serialization.py 全绿
- ☐ M4 端到端: CPU 上训一个极小模型 (d4-d8), loss 稳定下降, 并保存/加载 checkpoint 续训成功
- ☐ M5 R4 反向传播验证: 与 PyTorch autograd 的数值梯度对照 (相对误差 < 1e-6)
- ☐ M6 三组改坏实验 (去 mask / 换位置编码 / 去残差): 每组先写预测, 再跑, 再记录
- ☐ M7 一页报告 + 知识卡 ≥ 3 张

五、交付物

交付物	位置	验收
实现	手写/ (你写)	M2/M3 判分全绿

交付物	位置	验收
判分输出	判卷记录.md	证据/judge-*.log 原文
改坏实验记录	证据/改坏实验.md	预测→现象→解释 三段齐全
loss 曲线	证据/	训练日志 + 图（含坐标轴与步数）
报告	报告.md	一页四段

六、讲课锚点（讲义 M3 + 对标）

- **讲义**：讲义/M3-Transformer 内部机制.md（主干）+ 讲义/M0-数学与机器学习地基.md（§ 一 自动微分、§ 五 排查顺序）
- **对标**：CS336 lecture 03–04 · rasbt ch3–ch4 · 李沐 d2l ch10–ch11
- **必备工具链**：bash 环境/bootstrap_cpu.sh --with-torch（本课题起需要 torch）

七、代码级提问示例（讲课中出，先预测后验证）

1. 把因果 mask 从 softmax 之前挪到之后，训练现象会怎样？
2. 去掉残差后，4 层还能训、12 层完全不动——请用梯度路径解释。
3. 你把 RoPE 的最大频率底数从 10000 改成 100：短文训练会受影响吗？长上下文外推呢？
4. AdamW 的 weight_decay 若改成“直接加到梯度上”（即 L2），在大学习率下会出现什么差异？

八、风险与提醒

- ⚠ **CPU 训练极慢**：本课题只要能训起来、loss 下降即可，不追求指标（真训练在课题 04）。
- ⚠ **形状错误是最高频故障**：先写形状注释再写实现，可省一半时间。
- ⚠ **判分器需要 torch**；bootstrap --with-torch 首次约 555MB（沙箱网络受限时可能失败，届时改用 Kaggle CPU 会话）。